CETP-UTU · CTT · Módulo III

Técnico en Redes y Software

REDES POSIX — SERVICIOS

Planificaciòn Teórico-Práctica — systemd, Servicios de Red y Contenedores Docker

PERFIL A

Sin conocimientos previos

Conocés Bash scripting y SSH del S2. Ahora configurás Apache, Samba, firewall y usás Docker para desplegar servicios.

PERFIL B

Con conocimientos previos

Administrás Linux. Profundizás en virtual hosts, Docker Compose, SELinux, Dockerfiles multistage y hardening de contenedores.

Año Lectivo 2026 · Semestre 3· 16 semanas · 3 Unidades

Presentación

Propósito

Instalar, configurar, administrar y asegurar servicios en sistemas POSIX, comprendiendo systemd, servicios de red tradicionales y contenedores para ejecución aislada. Prepara para la Tutoría POSIX del S4.

Resultado: Administra y configura servicios POSIX con systemd, implementa servicios de red, aplica firewall y seguridad, y gestiona contenedores Docker para despliegue aislado de aplicaciones.

Sem.	Unidad	Hs.	Foco
1–5	U1: Gestión de servicios con systemd	~15	Daemons, units, journalctl, servicios propios
6–10	U2: Servicios de red — Apache, Samba, SSH avanzado	~15	Virtual hosts, Samba AD, UFW/firewalld
11–16	U3: Contenedores con Docker	~18	Docker, Dockerfile, Compose, hardening

U1

Semanas 1–5

Gestión de Servicios con systemd

Daemons, units, journalctl, crear servicios personalizados.

Saberes estructurantes y asociados	Saberes de profundización
– Daemon vs proceso vs servicio — diferencias – systemd: arquitectura y concepto de unit – systemctl: start, stop, restart, enable, disable, status, mask – Tipos de unit: service, socket, timer, target – Directorio /etc/systemd/system — units personalizadas – journalctl: -u, -f, --since, -p err – Rotación de logs: logrotate + journald – Crear service unit personalizada – Diagnóstico de fallos: systemctl status + journalctl	– Límites de recursos en units: CPUQuota, MemoryMax – Timer units — alternativa moderna a cron – Dependencies: Wants, Requires, After, Before – Systemd-analyze blame — ver tiempos de arranque

Marco teórico — Anatomía de una service unit

Una **unit file** es un archivo de configuración que describe un servicio a systemd. Se ubica en `/etc/systemd/system/` y tiene extensión `.service`.

Ejemplo — servicio de monitoreo CTT:

```
# /etc/systemd/system/ctt-monitor.service
[Unit]
Description=CTT System Monitor
Documentation=https://ctt.edu.uy/docs/monitor
After=network.target syslog.target
Wants=network-online.target

[Service]
Type=simple
User=ctt-monitor                # No ejecutar como root
Group=ctt-monitor
WorkingDirectory=/opt/ctt-monitor
ExecStart=/usr/local/bin/ctt_monitor.sh
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure              # Reiniciar si falla
RestartSec=10                   # Esperar 10s antes de reiniciar
StandardOutput=journal          # Salida al journal de systemd
```

```
StandardError=journal
# Hardening básico:
NoNewPrivileges=true            # No puede escalar privilegios
ProtectSystem=strict            # /usr, /boot, /etc son solo lectura
PrivateTmp=true                 # /tmp propio — aislado

[Install]
WantedBy=multi-user.target
```

Activar y gestionar el servicio:

```
systemctl daemon-reload              # Cargar la nueva unit
systemctl enable --now ctt-monitor   # Activar e iniciar
systemctl status ctt-monitor         # Verificar
journalctl -u ctt-monitor -f         # Log en tiempo real
```

PERFIL A — sin conocimientos previos

Gestión guiada de servicios:

- systemctl list-units --type=service --state=running — contar servicios activos
- journalctl --since '1 hour ago' -p err — errores de la última hora
- systemctl stop nginx → verificar que el estado es 'dead'
- systemctl mask apache2 — impedir que se pueda iniciar (accidental)
- Crear un service unit para el script ctt_monitor.sh del S2

PERFIL B — con conocimientos previos

Timer units y hardening:

- Timer unit: ctt-backup.timer que ejecuta ctt-backup.service cada día a las 2 AM
- Comparar timer vs crontab — ventajas de los timers de systemd
- Agregar hardening a un service unit existente (nginx): ProtectHome=true, ReadWritePaths=/var/log/nginx
- systemd-analyze blame — identificar los 5 servicios que más tardan al arrancar
- Optimizar: deshabilitar servicios innecesarios → medir el tiempo de boot

📋 EVALUACIÓN

Instrumento Perfil A

- Identificar qué falta en una unit file dada para que el servicio se inicie automáticamente
- ¿Cuál es la diferencia entre Restart=on-failure y Restart=always?

Instrumento Perfil B

- Crear service + timer unit para un script de backup con hardening
- Diagnóstico: servicio que falla al arrancar — usar journalctl para identificar la causa

Criterios de evaluación

30% Unit file correcta con sección [Unit][Service][Install]
25% Hardening básico en la unit
25% Timer unit funcionando
20% Diagnóstico con journalctl documentado

U2 Servicios de Red POSIX

Semanas 6–10

Apache/Nginx con virtual hosts, Samba, SSH avanzado y firewall.

Saberes estructurantes y asociados	Saberes de profundización
– Apache/Nginx: instalación, configuración básica, test – Virtual hosts: alojar múltiples sitios en un servidor – Samba: compartir recursos con Windows – NFS: compartir recursos entre sistemas Linux – SSH avanzado: ProxyJump, tunelización, agent forwarding – UFW/firewalld: reglas, zonas, servicios – SELinux/AppArmor: modos y troubleshooting	– Nginx como reverse proxy para Apache – SSL/TLS con Let's Encrypt (certbot) – Samba como Domain Member – NFS automount con autofs – fail2ban integrado con servicios web

Marco teórico — Virtual hosts y Samba

Apache — Virtual Host para portal.ctt.edu.uy:

```
# /etc/apache2/sites-available/portal.ctt.conf
<VirtualHost *:80>
    ServerName portal.ctt.edu.uy
    ServerAdmin admin@ctt.edu.uy
    DocumentRoot /var/www/portal
    ErrorLog ${APACHE_LOG_DIR}/portal-error.log
    CustomLog ${APACHE_LOG_DIR}/portal-access.log combined
    <Directory /var/www/portal>
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
a2ensite portal.ctt.conf && systemctl reload apache2
```

Samba — compartir carpeta con Windows:

```
# /etc/samba/smb.conf — agregar sección:
[Materiales]
    comment = Materiales CTT
    path = /srv/samba/materiales
    valid users = @docentes @alumnos
    read list = @alumnos
    write list = @docentes
```

```
    create mask = 0664
    directory mask = 0775
testparm && systemctl restart smbd
smbpasswd -a alumno1   # Crear contraseña Samba para el usuario
```

UFW — reglas esenciales:

```
ufw default deny incoming && ufw default allow outgoing
ufw allow 22/tcp  # SSH
ufw allow 80/tcp  # HTTP
ufw allow 443/tcp # HTTPS
ufw allow from 192.168.10.0/24 to any port 139,445  # Samba solo desde LAN
ufw enable && ufw status verbose
```

🧪 LABORATORIO PRÁCTICO
Servidor de servicios CTT completo (90 min)

1. Apache con 2 virtual hosts: portal.ctt.edu.uy y admin.ctt.edu.uy
2. Samba: compartir /srv/materiales (docentes escritura, alumnos lectura)
3. UFW: solo SSH, HTTP, HTTPS, Samba desde la LAN — denegar todo lo demás
4. Desde Windows: acceder a \\IP\Materiales con credenciales de docente y alumno
5. Verificar permisos: alumno puede leer, no puede escribir
6. curl http://portal.ctt.edu.uy — verificar que responde con el sitio correcto
7. tail -f /var/log/apache2/portal-access.log — ver accesos en tiempo real

📋 EVALUACIÓN

Instrumento Perfil A

- Configurar un virtual host que redirige HTTP → HTTPS
- Identificar el error en una configuración Samba que bloquea a todos los usuarios

Instrumento Perfil B

- Servidor con 2 virtual hosts + Samba + UFW correctamente hardened
- fail2ban configurado para Apache: bloquear IPs con más de 10 intentos 404 en 1 hora

Criterios de evaluación

25% Virtual hosts funcionando en Apache
30% Samba con permisos diferenciados
25% UFW con reglas mínimas necesarias
20% Logs y monitoreo documentados

U3 Contenedores con Docker

Semanas 11–16

Docker, Dockerfile, docker-compose y hardening básico de contenedores.

Saberes estructurantes y asociados	Saberes de profundización
– Contenedor vs VM — diferencias fundamentales – Docker: arquitectura, instalación, daemon – docker pull, run, ps, stop, rm, exec, logs – Imágenes: layers, tags, docker images, docker inspect – Volúmenes y bind mounts — persistencia de datos – Redes Docker: bridge, host, user-defined – Dockerfile: FROM, RUN, COPY, EXPOSE, CMD, ENTRYPOINT – docker-compose — orquestar múltiples servicios – Contenerizar Apache/Nginx – Hardening básico: usuario no-root, capacidades mínimas	– Multistage Dockerfile — imagen más ligera – docker-compose con healthchecks – Secrets en Docker Compose – Docker en producción: restart policies, resource limits – Integración con systemd: contenedor como servicio

Marco teórico — Docker en profundidad

Dockerfile para la aplicación web CTT (PHP + Apache):

```
FROM php:8.3-apache AS base

# Instalar extensiones PHP necesarias
RUN docker-php-ext-install pdo pdo_mysql && \
    a2enmod rewrite && \
    rm -rf /var/lib/apt/lists/*  # Limpiar cache de apt

# Copiar configuración de Apache
COPY docker/apache.conf /etc/apache2/sites-available/000-default.conf

# Copiar el código de la aplicación
COPY --chown=www-data:www-data . /var/www/html/

# Hardening: no ejecutar como root
USER www-data

EXPOSE 80
HEALTHCHECK --interval=30s CMD curl -f http://localhost/ || exit 1
```

docker-compose.yml — stack completo CTT:

```
version: '3.9'
services:
  web:
    build: .
    ports: ['8080:80']
    volumes: ['./:/var/www/html']  # bind mount para dev
    environment:
      DB_HOST: db
      DB_NAME: ctt
    depends_on:
      db: {condition: service_healthy}
    restart: unless-stopped

  db:
    image: mysql:8.0
    volumes: ['mysql_data:/var/lib/mysql']
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASS}  # desde .env
      MYSQL_DATABASE: ctt
    healthcheck:
      test: ['CMD','mysqladmin','ping','-h','localhost']
      interval: 10s

volumes:
  mysql_data:
```

Comandos esenciales docker-compose:

```
docker compose up -d          # Iniciar en background
docker compose down           # Detener y eliminar contenedores
docker compose logs -f web    # Log en tiempo real del contenedor web
docker compose exec web bash # Entrar al contenedor web
docker compose ps             # Estado de todos los servicios
```

PERFIL A — sin conocimientos previos

Docker básico — paso a paso:

- docker run -d --name nginx-test -p 8080:80 nginx — levantar Nginx en el puerto 8080
- docker exec -it nginx-test bash — entrar al contenedor
- docker logs nginx-test — ver los access logs de Nginx

PERFIL B — con conocimientos previos

Dockerfile optimizado y hardening:

- Multistage build: etapa 'builder' instala dependencias, etapa 'runtime' solo copia lo necesario
- Comparar tamaño: imagen con vs sin multistage (docker images)

- docker stop nginx-test && docker rm nginx-test — limpiar
- docker-compose up -d con el stack CTT (web + db) — verificar en http://localhost:8080

- Hardening: USER sin privilegios, --cap-drop=ALL --cap-add=NET_BIND_SERVICE
- docker scan o trivy para escanear la imagen en busca de vulnerabilidades
- Systemd unit para el docker-compose: reiniciar automáticamente si el host se reinicia

🧪 LABORATORIO PRÁCTICO

Proyecto final — Stack CTT contenerizado (evaluación final)

8. Dockerfile para la app CTT: PHP 8.3 + Apache, extensiones pdo_mysql, usuario non-root
9. docker-compose.yml con web + db (MySQL 8) + healthchecks + restart policies
10. Volumen named para persistencia de MySQL — los datos sobreviven a docker compose down
11. .env para las variables sensibles — .env en .gitignore
12. Bind mount del código fuente en desarrollo — hot reload al editar
13. docker compose up -d → verificar que la app CTT funciona completamente
14. Hardening: usuario non-root, ProtectSystem en systemd si se wrappea como servicio
15. README con instrucciones de inicio: prerequisitos, clonar, .env, docker compose up

📋 EVALUACIÓN

Instrumento Perfil A

- Escribir el Dockerfile para una app Node.js con usuario non-root
- ¿Por qué se usa depends_on con healthcheck en lugar de solo depends_on?

Instrumento Perfil B

- Stack completo con Dockerfile optimizado, docker-compose, .env y README
- Análisis de seguridad: trivy scan de la imagen — identificar vulnerabilidades y corregir 2

Criterios de evaluación

25% Dockerfile correcto con hardening

30% docker-compose con healthchecks y volúmenes

30% Stack funcionando end-to-end

15% README y .env correctamente configurados

Cierre del Curso

Lo que aprendiste

U1: systemd units, journalctl, crear servicios propios, timers, hardening
U2: Apache virtual hosts, Samba, UFW, SSH avanzado
U3: Docker, Dockerfile, docker-compose, volúmenes, hardening

- S4 — Tutoría POSIX: proyecto integrador de administración Linux completo

Kubernetes, la siguiente capa sobre Docker, está en el 80% de las ofertas de DevOps en Uruguay. Este curso es el punto de partida.